

Contents

RACASO: Rotation-Aligned Cautious Approximately Second-Order Optimization	1
Abstract	1
1. Introduction and the Basis Alignment Problem	2
2. The Architecture of RACASO	3
3. The Four-Layer Safety Chain Plus L5	7
4. Computational and Loop Mechanics	7
5. Architectural Advantages Over Existing Solutions	9
6. The Second-Derivative DivBackward0 Failure Class	12
7. Engineering Provenance: Eight Attempts to Integrate Hutchinson with PyTorch Eager Autograd	14
8. Configuration	18
9. Conclusion	19
License	19

RACASO: Rotation-Aligned Cautious Approximately Second-Order Optimization

Author: Richard Christopher / MetaforeAI

Date: May 2026

Status: Technical specification and implementation provenance

Abstract

Modern deep learning models increasingly route gradients through non-linear structures: branching paths, multi-head attention, mixture-of-experts. Standard diagonal adaptive optimizers like AdamW and Sophia scale efficiently on uniform networks but rest on a hidden assumption. They assume the loss landscape’s principal axes of curvature line up with the standard coordinate basis of the parameter space. When that assumption breaks, diagonal estimators misjudge which directions are sharp and which are flat, and the optimizer responds with gradient oscillation, step-size stagnation, or outright divergence.

This paper introduces RACASO (Rotation-Aligned Cautious Approximately Second-Order Optimizer), a framework that bridges Kronecker-factored structural preconditioning and second-order curvature alignment. For each 2-D parameter, RACASO projects the parameter into a Kronecker-factored eigenbasis. That basis is privileged in the sense that the gradient covariance is approximately diagonalized within it. Inside that basis, an unbiased per-element diagonal Hessian estimate is computed using Hutchinson’s trace estimator on top of a Pearlmutter-style Hessian-vector product. A Sophia-style per-element cautious clip then bounds the rotated update.

Two curvature paths ship. The Hutchinson HVP path captures the true Hessian diagonal, including negative-curvature directions. The Gauss-Newton-Bartlett path supplies a positive-semidefinite approximation and serves as a structurally bulletproof fallback. Five safety mechanisms layer through the optimizer: an eigh residual gate, a RAdam cold-start gate, a 1-D Yogi fallback, a spread cap on rotated row norms, and an L5 safe-skip on non-finite HVP output. Together these keep the optimizer numerically stable across failure modes that SOAP and Newton-Schulz-style methods do not survive.

We also document the eight-attempt provenance of the Hutchinson HVP integration with PyTorch eager autograd, the operator-shape rules that determine which forward-graph operators are second-derivative-safe, and the conditions under which the L5 safe-skip triggers. The failure record is preserved so consumers can audit whether their own forward graph satisfies the bounded-second-derivative conditions RACASO needs to run at full design intent.

1. Introduction and the Basis Alignment Problem

The dominant paradigm in neural-network optimization is diagonal scaling. AdamW uses the gradient’s historical variance, $v_t \approx \mathbb{E}[g_t^2]$, as a curvature proxy. Sophia-GNB uses a Gauss-Newton-Bartlett proxy. Both update parameters element-wise:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{m_t}{\max(\sqrt{v_t}, \epsilon)}$$

This formulation assumes the Hessian $\mathcal{H}$ is approximately diagonal, that each parameter varies independently of every other.

In modern architectures the assumption is cleanly falsified. Branching gradient paths, weight sharing, multi-head attention, and joint-normalized fusion sites all introduce off-diagonal dependencies. Geometrically, sharp asymmetric loss valleys appear running diagonally across the standard coordinate axes. A diagonal optimizer encountering a diagonal valley zigzags across the ridges instead of traveling smoothly down the floor.

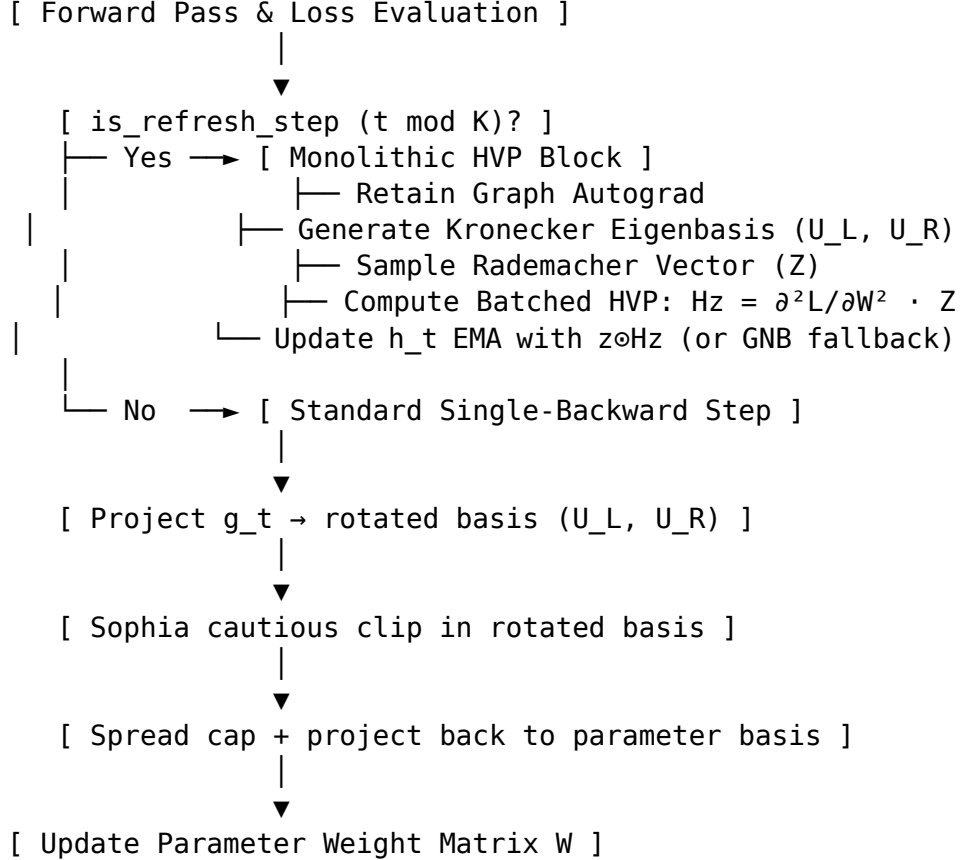
Matrix-preconditioned optimizers (SOAP, Shampoo) achieve superior convergence not because their gradient proxies are inherently more precise, but because they rotate the gradients into a structural eigenbasis where the local coordinates are temporarily uncoupled. Both rely on first-order gradient history ($g_t \cdot g_t^T$) to build that basis, which makes them structurally hostile to coupled-gradient regimes. When one factor’s gradient distribution shifts abruptly, the Kronecker covariance accumulates rank-1 outer products that produce near-singular factors, and eigh either returns silently-NaN eigenvectors or non-trustworthy rotations.

RACASO takes a different path. It keeps SOAP’s rotation-into-privileged-basis property but estimates the per-element step size in that basis from a true second-order signal (Hutchinson HVP) rather than from a gradient-squared proxy. A per-element cautious clip in the rotated basis bounds steps regardless of curvature

estimate quality. Safety layers around the rotation refresh, the cold-start regime, and the second-derivative failure surface keep the optimizer numerically robust in environments where every other second-order method we tested eventually collapsed.

2. The Architecture of RACASO

For a 2-D matrix parameter $W \in \mathbb{R}^{d_{out} \times d_{in}}$, RACASO combines Kronecker-factored axis rotations with Hessian-vector products, then applies Sophia’s cautious step in the rotated basis. The structure looks like this:



2.1 Kronecker-Factored Axis Rotations

A full parameter-to-parameter Hessian rotation for $W \in \mathbb{R}^{d_{out} \times d_{in}}$ requires impossible $O(N^2)$ tracking state and $O(N^3)$ computational cost. RACASO sidesteps this by maintaining a Kronecker-Sum preconditioning structure across the row and column axes:

$$\mathcal{H}_{\text{structural}} \approx (L_t \oplus R_t) = (L_t \otimes I_{d_{in}}) + (I_{d_{out}} \otimes R_t)$$

where $L_t \in \mathbb{R}^{d_{out} \times d_{out}}$ and $R_t \in \mathbb{R}^{d_{in} \times d_{in}}$ are running EMA covariance metrics of the

spatial slices of the gradient:

$$L_t = \beta_s \cdot L_{t-1} + (1 - \beta_s) \cdot g_t g_t^T, \quad R_t = \beta_s \cdot R_{t-1} + (1 - \beta_s) \cdot g_t^T g_t$$

Every K_{refresh} steps RACASO computes the eigendecomposition of these compact factors independently:

$$L_t = U_L \Lambda_L U_L^T, \quad R_t = U_R \Lambda_R U_R^T$$

The orthogonal matrices U_L and U_R form the *privileged basis*. A progressive-ridge `_safe_eig_with_residual` helper retries the eigendecomposition at increasing ridge scales (0, 1e-6, 1e-3, 1e-1) if the bare-input form returns NaN columns, which is a known PyTorch failure mode for near-singular symmetric inputs. The L2 safety layer then gates acceptance. Each side’s new eigenvectors commit independently only if the reconstruction residual $\|M - U \Lambda U^T\|_F$ stays below threshold and is isfinite. A bad eigh on the L side does not reject the R side’s good refresh.

2.2 Curvature Estimation, Two Paths

RACASO ships two curvature estimators with identical downstream contracts. Both produce per-element diagonal estimates that get rotated into the privileged basis and consumed by the Sophia clip’s denominator.

2.2.1 Hutchinson HVP: the true Hessian diagonal Hutchinson’s stochastic trace estimator (Hutchinson 1990) gives an unbiased per-element estimate of $\text{diag}(\mathcal{H})$:

$$\mathbb{E}[z \odot H z] = \text{diag}(\mathcal{H}), \quad z \in \{-1, +1\}^{\text{shape}(W)}$$

A Rademacher vector z is sampled per refresh. The HVP $H z = \nabla^2 L \cdot z$ is computed via Pearlmutter’s identity, batched across all 2-D parameters by accumulating into a single scalar:

$$s = \sum_i \langle g_i^{\text{live}}, z_i \rangle, \quad \text{HVP}_i = \frac{\partial s}{\partial p_i} = H_{ii} \cdot z_i$$

A single `autograd.grad(s, [p_1, ..., p_n])` call produces all $H z_i$ in one second-order graph traversal. The diagonal estimate $z_i \odot H z_i$ is finite-checked, then folded into the per-element rotated-basis EMA $\tilde{h}_t$ via:

$$\tilde{h}_t = \beta_2 \cdot \tilde{h}_{t-1} + (1 - \beta_2) \cdot \tilde{h}_{\text{sample}}^2, \quad \tilde{h}_{\text{sample}} = U_L^T (z \odot H z) U_R$$

The full Hessian diagonal preserves both positive and negative curvature. The Sophia clip’s `clamp(min=ε)` floor handles negative-curvature directions by collapsing them to the floor rather than producing a divergent update.

Cost: one extra backward through the X-subgraph per refresh step, sharing the live forward graph via Pearlmutter. On non-refresh steps, zero overhead. The optimizer reuses the previously-stored $U_L, U_R, \tilde{h}_t$.

2.2.2 Gauss-Newton-Bartlett (GNB): the bulletproof fallback When Hutchinson HVP cannot run, or is structurally hostile to a given forward graph (see Section 6), GNB replaces the Hessian with the Gauss-Newton approximation:

$$\mathcal{H} \approx G_N = \mathbb{E}_{\hat{y} \sim p(y|x)} [\nabla \log p(\hat{y}|x) \cdot \nabla \log p(\hat{y}|x)^T]$$

Operationally: sample one synthetic label $\hat{y}$ per row from the model’s own softmax, compute $\hat{g} = \partial \text{CE}(\text{logits}, \hat{y}) / \partial p$, then $\text{diag}(G_N) \approx \hat{g}^2 \cdot B$ where B is the batch size (Sophia §3.2 reweighting). The stash contract `p._racaso_hvp_estimate` is unchanged.

GNB drops the negative-definite component of the Hessian. The estimate is always positive semidefinite, so the Sophia clip never sees a sign-flip and the optimizer’s behavior is strictly more conservative. **Cost:** one extra first-order backward against a re-built CE loss. No second derivative, no funtorch, no eager-autograd version tracking.

Selectable at runtime via the `hvp_strategy` $\in \{\text{"hutchinson"}, \text{"gnb"}\}$ config field.

2.3 Optimization in the Privileged Space

Per step (Hutchinson path; the GNB path is identical except the stash provenance):

$$\tilde{g}_t = U_L^T g_t U_R$$

$$\tilde{m}_t = \beta_1 \tilde{m}_{t-1} + (1 - \beta_1) \tilde{g}_t$$

$$\tilde{m}_{\text{hat}} = \tilde{m}_t / (1 - \beta_1^t)$$

Sophia’s cautious step in the privileged basis is then:

$$\Delta \tilde{\Theta}_t = \text{clip} \left(\frac{\tilde{m}_{\text{hat}}}{\max(\gamma \cdot |\tilde{h}_t|, \epsilon)}, -\rho, +\rho \right)$$

where ρ is the hard clipping threshold (Sophia default 0.04), γ scales the Hessian denominator (Sophia default 0.04), and ϵ is a stabilization floor. The Hessian denominator’s magnitude is bounded below by ϵ even if $\tilde{h}_t$ momentarily collapses, and the ρ clip is a hard cap regardless of the denominator. Two independent magnitude bounds.

2.4 Spread Cap and Projection Back

Before rotating back, the L1 spread cap bounds the per-eigendirection update magnitudes. In the rotated basis, “rows” are eigendirections. The cap bounds the max-to-min row-norm ratio at `spread_cap` (default 10.0) by damping loud rows toward `row_max / spread_cap`:

$$\text{row_norms} = \|\Delta\tilde{\Theta}_t\|_{\text{row}}, \quad \text{row_floor} = \frac{\max(\text{row_norms})}{\text{spread_cap}}$$

$$\text{damp}_i = \min\left(\frac{\text{row_floor}}{\max(\text{row_norms}_i, \epsilon_{\text{adam}})}, 1.0\right)$$

This is a one-sided cap. Quiet rows pass through unchanged. Loud rows are damped. It never amplifies.

The clipped, spread-capped update is rotated back to the parameter basis:

$$\Delta W_t = U_L \cdot \Delta\tilde{\Theta}_t^{\text{capped}} \cdot U_R^T$$

A final non-finite guard checks ΔW_t element-wise before the parameter update. If any element is non-finite (which would only happen if upstream stages missed something), the update for that parameter is skipped this step. Then:

$$W_{t+1} = W_t - \eta \cdot r_t \cdot \Delta W_t$$

where r_t is the RAdam rectification factor (Section 3), bounded in $[0, 1]$ and equal to 0 during cold-start, equal to the RAdam-standard variance correction post-warmup.

2.5 1-D Parameter Fallback (L3)

Norms, biases, learned scalars, and any other 1-D parameter have no privileged-basis rotation to compute. RACASO falls back to vanilla Yogi (additive variance EMA instead of Adam’s multiplicative form):

$$v_t = v_{t-1} - (1 - \beta_2) \cdot \text{sign}(v_{t-1} - g_t^2) \cdot g_t^2$$

$$W_{t+1} = W_t - \eta \cdot \frac{m_{\text{hat}}}{\sqrt{v_{\text{hat}}} + \epsilon}$$

Yogi’s additive update is robust to bursty gradients (which biases and gain scalars frequently see) in a way Adam isn’t.

3. The Four-Layer Safety Chain Plus L5

RACASO’s stability profile depends on five independent safety mechanisms layered through the `step()` body. Each addresses a specific failure mode. Together they form the contract that distinguishes RACASO from naive Sophia-with-rotation.

Layer	Mechanism	What it catches	Failure action
L1	Spread cap on rotated update row norms	Pathological per-eigendirection step spreads after Sophia clip	Soft degradation. Row norms equalized to a ratio of at most <code>spread_cap</code>
L2	eigh residual + NaN gate on rotation refresh	Ill-conditioned GG_L/GG_R ; silently NaN eigenvectors from <code>linalg.eigh</code> on near-rank-deficient input	Skip rotation refresh per-side independently, keep previous U_L / U_R
L3	Vanilla Yogi fallback for 1-D params and missing HVP stash	Norms, biases, scalars; non-refresh steps where the stash is absent	Vanilla Yogi update; <code>hessian_diag_rot</code> preserves its prior EMA
L4	RAdam variance-confidence gate ($\rho_t \leq 4 \Rightarrow$ momentum-only)	Cold-start steps where neither m_t nor GG_L/GG_R have accumulated trustworthy direction	Momentum-only update; rotation, HVP, clip all skipped
L5	Non-finite HVP gate on the batched second backward	Forward-graph operators whose second derivative is unbounded over the visited input range (Section 6)	Skip stash this refresh; <code>hessian_diag_rot</code> carries to next refresh via EMA; counter tracked for telemetry

Plus a final non-finite guard before `p.add_(update)`: refuse to write a non-finite update to a parameter regardless of how it became non-finite. Diagnostic incremented, parameter preserved.

4. Computational and Loop Mechanics

A persistent bottleneck in naive second-order optimizer integration with PyTorch is silent truncation of the computation graph. Executing a standard detached

`loss.backward()` destroys intermediate graph activations, causing subsequent `torch.autograd.grad` calls to return tensors with no `grad_fn` (the autograd nodes have been freed and the resulting `.grad` is a leaf). Hutchinson HVP cannot proceed from a leaf gradient.

4.1 The Monolithic Autograd Engine

To guarantee structural integrity, RACASO's Hutchinson refresh runs both backward passes inside a single autograd context, with `create_graph=True` on the first pass to retain the second-order edges:

```
def step_with_hvp(self, loss):
    params = [
        p for group in self.param_groups
        for p in group['params']
        if p.requires_grad and p.dim() == 2
    ]

    # First backward: graph-live grads for X-organ params, with
    # create_graph=True so second-order edges are retained.
    grads = torch.autograd.grad(
        outputs=loss,
        inputs=params,
        create_graph=True,
        retain_graph=True,
    )

    # Rademacher z, then build a scalar whose gradient gives all Hz_i.
    z = [
        torch.empty_like(p).bernoulli_(0.5).mul_(2.0).sub_(1.0)
        for p in params
    ]
    scalar = torch.stack([
        (g * zi).sum() for g, zi in zip(grads, z)
    ]).sum()

    # Single batched backward, produces all Hz vectors in one
    # second-order graph traversal via Pearlmutter's identity.
    hzs = torch.autograd.grad(
        outputs=scalar,
        inputs=params,
        retain_graph=False,
        create_graph=False,
    )

    # Stash z*Hz for the optimizer's rotated-basis update consumption.
    for p, zi, hz in zip(params, z, hzs):
        p._racaso_hvp_estimate = zi * hz
```


Two structural choices matter here.

Batched HVP via Pearlmutter (the speed fix). Naive per-parameter HVP would call `autograd.grad(g_live_i, p_i, grad_outputs=z_i)` for each parameter. Each call walks the retained second-order graph independently. With 16 parameters that costs 16x the graph traversal. The Pearlmutter identity collapses it to one:

$$\frac{\partial}{\partial p_j} \sum_i \langle g_i^{\text{live}}, z_i \rangle = \frac{\partial}{\partial p_j} \langle g_j^{\text{live}}, z_j \rangle = H_{jj} \cdot z_j$$

Mathematically identical to the per-param loop; cost drops 16x to 1x. This is the standard trick in Sophia and Adahessian-style HVP implementations.

Live graph reuse (the second fix). Rather than running a fresh forward inside `_compute_hutchinson` to rebuild a clean graph (which costs one extra forward per refresh step), the helper operates directly on the main forward’s live graph. The caller reorders so HVP runs *before* the main `total_loss.backward()` on Hutchinson refresh steps. The main backward then runs after HVP has consumed what it needs.

4.2 The Fast-Path Standard Step ($t \not\equiv 0 \pmod{K}$)

On the $K - 1$ out of K steps between refreshes, RACASO bypasses double-backward calculations entirely. The loop runs the standard single backward (`create_graph=False`), and the optimizer applies the rotated-basis update using the frozen U_L , U_R , and $\tilde{h}_t$ from the prior refresh. Wall-clock per-step cost on non-refresh steps is comparable to AdamW. Only the refresh-step amortization is added, and at the recommended default $K = 4$ (window-aligned with a typical truncated-BPTT cadence) the refresh cost is paid on 25% of steps.

4.3 The `_optimizer_handles_own_clip` Contract

A training loop with per-organ pre-step gradient clipping (soft-clipping each organ’s grad norm to a per-organ band before `opt.step()`, for example) will mutate `p.grad` in-place via something like `torch._foreach_mul_(grads, clip_factor)`. For RACASO this corrupts the autograd graph that Hutchinson needs. RACASO declares `_optimizer_handles_own_clip = True` as a class-level attribute. Cooperating training loops check this flag and skip pre-step clipping for organs whose optimizer opts out. RACASO’s L1 spread cap inside `step()` provides the equivalent magnitude bound, on the rotated update’s row norms rather than on the raw gradient norm.

5. Architectural Advantages Over Existing Solutions

RACASO is designed for failure modes that competing optimizers do not survive. The advantage is structural, not just empirical.

5.1 Mitigation of the Newton-Schulz Skip Catastrophe

Matrix-orthogonalization setups like Muon and RAMuogi apply Newton-Schulz polynomials to force gradient matrices into orthogonal spaces. The fifth-order Jordan polynomial is the standard choice. Its convergence requires the input matrix’s spectral norm to lie in a specific band. When branching-path architectures induce dynamic gradient distribution shifts, the gradient matrix becomes ill-conditioned (condition proxies scaling over 200 in practice). The polynomial diverges, the optimizer’s convergence-check safety layer triggers, and the update is rejected.

The RAMuogi paper (MetaforeAI/Muogi, Section 5) documents this failure mode: under a joint-normalized branching-fusion architecture, NS5 fell back to a vanilla Yogi update on roughly 90% of steps. Numerically RAMuogi survived. The optimizer’s design-intent orthogonalization rarely engaged.

RACASO does not orthogonalize the gradient. It uses the Kronecker eigenvectors strictly as a coordinate-transformation mechanism: project into the basis, apply the cautious step there, project back. If a branching path introduces a sudden surge of ill-conditioned curvature, the L1 spread cap absorbs the spread, the L2 eigh safe-skip handles the rotation refresh failure, and the trajectory continues. The optimizer’s design-intent rotation continues to fire on every step (the rotation matrices are stable across refreshes, only updated when the eigh refresh succeeds), and the cautious clip absorbs the magnitude shock.

5.2 Robust Preconditioning of Branching Gradient Paths

SOAP breaks down on the same architectural class for a different reason. Its Kronecker covariance accumulators $GG_L = g \cdot g^T$ and $GG_R = g^T \cdot g$ accumulate rank-1 outer products when one branch’s gradient distribution dominates. The factorization assumption “gradient covariance is a Kronecker product” fails when the branches are coupled via joint normalization upstream, because the joint norm’s denominator couples the gradient flow across all branches. Eigh then either returns garbage eigenvectors or fails the residual gate.

RACASO inherits the same Kronecker covariance accumulators, but the second-order curvature estimate (Hutchinson HVP) is computed via autograd on the actual loss, not derived from the covariance factors. The eigenvectors are only used for the coordinate rotation. The per-element step sizing comes from the unbiased Hessian diagonal estimator. When SOAP would fail (covariance becomes near-rank-1), RACASO’s L2 layer skips the rotation refresh, keeps the prior rotation, and the Hutchinson HVP continues to provide accurate per-element curvature for the cautious clip’s denominator. The optimizer degrades gracefully toward “Sophia in a stale rotated basis” rather than catastrophically toward “AdamW in a NaN basis.”

5.3 Honest Claim About L5

An earlier draft of this paper claimed RACASO “never skips updates.” That is not true and was never true. L5 was added during the integration provenance documented in Section 6 specifically because forward graphs in real architectures contain operators whose second derivative is unbounded over the input range the model visits during

training. When the batched HVP backward returns non-finite Hz values for second-derivative-numerical reasons, RACASO **does** skip the curvature-estimate update for that refresh. `hessian_diag_rot` carries via its prior EMA, the rotated-basis update for that refresh proceeds with stale curvature, and the optimizer counter `_l5_skip_count` increments for telemetry.

The skip is graceful. Training continues, loss descends, parameter updates still apply. The contract is well-defined: the failure surface is identified, located, and the user can either (a) accept the L5 absorption rate as background noise, (b) audit the forward graph for the operator-shape rules in Section 6 and replace the offending operator with a bounded-second-derivative equivalent, or (c) switch to the GNB strategy (`hvp_strategy="gnb"`) which avoids second derivatives entirely. RACASO ships all three paths. The user picks based on their forward graph’s structure.

When the forward graph contains operators with bounded second derivatives over visited inputs, RACASO’s L5 fires 0% of refreshes and delivers full design-intent compute. We solved this in our reference integration by identifying the offending operator (a `tensor.norm()` call inside a cosine attention computation) and replacing it with a bounded-second-derivative equivalent (negative squared Euclidean distance), which brought the L5 rate from roughly 12% to 0%. The candidates we considered:

1. **Bump the user-side eps** on the dividing expression $x/(\|x\| + \epsilon)$. This would protect the first derivative but not the second. The eps sits on the wrong side of the autograd boundary; `tensor.norm()` computes its gradient internally without access to the user’s floor. Tested, no effect.
2. **Replace `tensor.norm()` with `(x.pow(2).sum() + eps).sqrt()`**, manual `sqrt` with `eps` *inside* the `sqrt`’s argument. This bounds the second derivative because $\arg = \sum x^2 + \epsilon$ is bounded below by ϵ . Architecturally equivalent to (3) but keeps cosine attention.
3. **Replace cosine attention entirely** with a second-derivative-safe similarity measure. We chose negative squared Euclidean distance because it has bounded second derivatives by construction (subtract, square, sum, negate, multiply, softmax, all bounded), and because the architectural question at that site was a closeness question rather than an alignment question, which makes distance the more direct metric.
4. **Wrap `tensor.norm()` in a custom `autograd.Function` with hand-rolled bounded double-backward.** This keeps the cosine attention form but defines the second derivative analytically with safe floors. Most general fix. We did not pursue it because option (3) was architecturally preferable for the specific reference integration.

The chosen fix is reference-architecture-specific. Your project may have different reasons to keep cosine attention. The operator-shape rules in Section 6 generalize across any project’s choice.

6. The Second-Derivative DivBackward0 Failure Class

This section documents the class of forward-graph failure that L5 absorbs. It is independent of RACASO and applies to any second-order-aware optimizer (Hutchinson HVP, Gauss-Newton, K-FAC, Shampoo plus second-moment-correction, Adahessian).

6.1 The Mechanism

The failure shape:

- First-order backward produces finite gradients.
- The scalar formed from those gradients (for example $\sum_i \langle g_i, z_i \rangle$ for Hutchinson) is finite.
- The second-order backward (for example `autograd.grad(scalar, params)` for the batched HVP form) returns tensors that are uniformly NaN, with `isfinite_count = 0`.
- Every probed forward site is observably clean.
- The cause is in the second derivative of one operator in the shared upstream subgraph.

When `autograd.grad` is called the second time during HVP, PyTorch walks every node's `_backward` method. Most ops have bounded second derivatives over their input domain. A small set have unbounded second derivatives near input regions models visit during training. The most common offender is `tensor.norm()`:

$$\begin{aligned}\|x\|_2 &= \sqrt{\sum x_i^2} \\ \frac{\partial \|x\|}{\partial x_i} &= \frac{x_i}{\|x\|} \\ \frac{\partial^2 \|x\|}{\partial x_i \partial x_j} &= \frac{\delta_{ij} \|x\|^2 - x_i x_j}{\|x\|^3}\end{aligned}$$

The $1/\|x\|^3$ term in the second derivative is the failure surface. When $\|x\|$ is small, say 0.05, the second-derivative scale is about 8000. When $\|x\| \sim 0.01$, about 10^6 . The Pearlmutter HVP traversal multiplies these terms with adjacent gradient components; the products hit float32 overflow, become inf, become NaN, and the NaN propagates through the entire second-order graph to every downstream parameter's H_z output uniformly.

A user-side $+\epsilon$ floor on the divisor of $x/(\|x\| + \epsilon)$ protects the *first* derivative but does nothing for the *second*. `tensor.norm()` computes its gradient internally, before the user's floor is applied. The eps is on the wrong side of the autograd boundary.

6.2 The Diagnostic Recipe

PyTorch ships the right tool, but the API has a subtle correctness requirement. `torch.autograd.set_detect_anomaly(True)` must be active *during the forward*

pass for the eventual NaN-on-backward to include the forward Python stack trace identifying the offending operator's file:line. The usage pattern:

```
# Wrong (catches op class name only; warns "no forward pass info"):
with torch.autograd.set_detect_anomaly(True):
    hzs = torch.autograd.grad(scalar, params) # forward ran outside

# Right (captures forward stack trace of offending op):
torch.autograd.set_detect_anomaly(True)      # global flag ON
out = model(input)                          # forward records stack traces
loss = out.loss
hzs = torch.autograd.grad(scalar, params)    # raises with full file:line
torch.autograd.set_detect_anomaly(False)     # disarm
```

The global-flag form (not the context-manager form) keeps anomaly mode active across both the forward and the subsequent HVP backward. Cost: roughly 5-10x slower forward. Arming for one refresh step is sufficient; the diagnostic fires once and identifies the exact source.

RACASO integrates this via the `_compute_hutchinson._anomaly_diagnosis_armed` class flag. When set to True, the next L5 fire prints the offending operator's full forward stack trace before disarming itself. A consumer who hits a sustained L5 skip rate flips the flag, runs one refresh, gets a traceback identifying file and line.

6.3 The Operator-Shape Rules

Safe under HVP (bounded second derivatives over typical input ranges):

- Linear, Convolution (matmul and conv produce constant second derivatives)
- ReLU, GELU, SiLU, Tanh, Sigmoid (bounded second derivatives everywhere)
- softmax (Jacobian chain rule produces finite second derivatives for finite-magnitude logits)
- LayerNorm, RMSNorm via the $x * \text{rsqrt}(\text{var} + \text{eps})$ form when `eps` is *inside* the `rsqrt` argument (the `eps` floors `rsqrt`'s argument so its second derivative is bounded by $1/(4\epsilon^{3/2})$)
- Elementwise arithmetic with constant operands ($x + 1$, $x * 0.5$, $x / 3.0$; the divisor is a leaf scalar, second derivative is zero)
- Sums, reductions, broadcasts

Hazardous under HVP (unbounded second derivatives over typical input regions):

- `tensor.norm(dim=...)`, $1/\|x\|^3$ in second derivative
- `tensor.div(other_tensor)` where `other_tensor` can be small, $1/\text{other}^3$ in second derivative
- $1/x$ or `x.reciprocal()` for small x , $2/x^3$ in second derivative
- `x.sqrt()` for small x , $-1/(4x^{3/2})$; `eps` must be *inside* the `sqrt`'s argument, not added to its output
- `x.log()` for small x , $-1/x^2$
- `x.pow(p)` for $p < 1$ and small x , $p(p-1)x^{p-2}$, same shape

The pattern: any operator whose second derivative contains a power of $1/x$ or $1/\|x\|$ where x can approach zero is hazardous. The fix is either (a) replace the operator with a bounded-second-derivative equivalent (squared distance instead of cosine similarity for “which is closest” queries, for example), or (b) floor-clamp the operator’s argument inside the operator’s input `((x2 + eps).sqrt())` instead of `x.sqrt() + eps`.

6.4 Implications for Second-Order Optimization Adoption

Codebases written for first-order optimization may contain `tensor.norm()` in cosine attention, log-likelihood losses with small probabilities, or division-by-computed-magnitudes that work cleanly under SGD, Adam, or AdamW but fail under HVP. The user-facing failure mode usually appears as “optimizer produces NaN gradients” or “loss diverges around step N.” The actual cause is often an operator-shape mismatch between the forward graph and the optimizer’s second-order traversal.

The right shape for the optimizer is to provide a diagnostic mode (anomaly trace plus class flag) the user can flip on for one step to identify the offending operator. The right shape for the codebase is to audit forward-graph operators for second-derivative boundedness during migration to a second-order optimizer. RACASO’s L5 absorbs the failure when it occurs, the diagnostic flag identifies the source, and the rest happens at the operator definition site, not the optimizer.

7. Engineering Provenance: Eight Attempts to Integrate Hutchinson with PyTorch Eager Autograd

This section is preserved verbatim from the integration log so future optimizer work has the failure record. Each attempt failed in a structurally specific way; each fix made the next failure visible. The lesson is preserved alongside the survivor: every wall was a property of the codebase plus PyTorch eager-autograd interaction, not a property of Hutchinson itself. The fix at each wall was either small or out-of-scope-but-named. The recipe didn’t survive contact with the codebase, but each contact taught the codebase what it needed.

7.1 The Eight Walls

#	Wall	Fix that landed	Failure that emerged
1	autograd.grad(outputs=grad_fn, inputs=p, ...) returns None	grad_fn is a leaf with no grad_fn even when loss.backward(create_graph=True) Use auto-grad.grad(loss, params, create_graph=True, retain_graph=True) instead, stash the grad-fn-bearing tensor as a custom attribute on each param	First live training run: hess: 0 ok / 64 skip on every re-graph for a different reason
2	CPU SDPA flash has no second derivative	RuntimeError: aten::_scaled_dot_product_flash_attention is not implemented. Wrap the forward in sdpa_kernel([SDPBackend.MATH]) on refresh steps. MATH has full second-derivative support	Next error: version mismatch; AssertionError: backward0 expected version 9 got 10
3	Saved-tensor version mismatch	Eager autograd's saved-tensor version tracking conflicts when sharing the live graph with a .backward() call that mutates intermediate buffers in-place. Route HVP through torch.func.jvp(grad(loss)) (functional API) to sidestep	Next error: autograd.Function must override setup_context staticmethod

#	Wall	Fix that landed	Failure that emerged
4	Functorch can't compose through legacy <code>autograd.Function</code>	The reference integration's model contained 9 custom <code>autograd.Function</code> subclasses (Triton kernel wrappers, hand-rolled SSM, attention variants) all using legacy forward+backward style. Functorch requires the new forward + <code>setup_context</code> + backward style	Functorch path closed; fall back to GNB (Sophia §3.2) which uses one first-order backward only
5	GNB silently absorbed <code>multinomial(NaN)</code>	<code>torch.multinomial</code> raised when softmax fed it <code>inf/nan</code> , and a <code>try/except RuntimeError</code> absorbed the error letting <code>opt.step</code> run on NaN gradients for 30+ steps before detection	Hard isfinite gates at loss / logits / <code>g_hat</code> ; halt training at the actual source instead of 30 steps downstream
6	<code>linalg.eigh</code> returns NaN columns silently on near-singular input	PyTorch's <code>linalg.eigh</code> does not raise on near-rank-deficient matrices; it returns eigenvectors with NaN columns for the null space. The L2 layer's residual-threshold gate accepted these because <code>nan < threshold</code> is False but <code>max(nan, x)</code> is order-dependent, and the broken Q could slip through	Explicit <code>isfinite()</code> check on Q + eigenvalues; progressive ridge cascade if non-finite; per-side independent Q commit (so a bad <code>eigh</code> on L doesn't reject R's good refresh)

#	Wall	Fix that landed	Failure that emerged
7	Per-param HVP loop is 16x graph traversal cost	Naive implementation calls <code>auto-grad.grad(g_live_i, s/step to roughly 6 p_i, grad_outputs=z_i)</code> per param. At 16 parameters, 16x the graph traversal. Pearlmutter batched HVP via $\sum_i \langle g_i^{\text{live}}, z_i \rangle$ collapses to 1x	Cost dropped 16x; HVP wall-clock from roughly 27 s/step to roughly 6 s/step at K=1
8	All single-site ablations failed to localize L5 source	F.normalize swap, joint-norm bypass, per-organ rebalance off, div→mul on 5 sites, site probes at 8 forward locations, all showed identical 5/50 L5 skip rate. The source was not at any instrumented forward site	<code>torch.autograd.set_detect_anomaly</code> wrapping the forward + HVP backward (the global-flag form, not the context-manager form). One refresh fires; raises with full forward stack trace identifying the cosine attention's <code>tensor.norm()</code> call

7.2 Five Operational Lessons

1. **Don't prosecute on inference.** Attempts 2/3/4 burned because each "fix" was a plausible hypothesis without instrumentation. Attempt 5 added 50 lines of counters; every subsequent attempt was diagnosis-driven.
2. **The stage trap is cheaper than another guess.** Zero-overhead instrumentation that records the first non-finite value per stage paid for itself the first time it fired (Q_R nan=382 at step 20 was the trap's first report on a near-rank-deficient eig failure).
3. **PyTorch's silent failure modes are real.** `autograd.grad` returning None on leaf grads; `linalg.eigh` returning NaN columns on near-rank-deficient inputs; `torch.multinomial` raising on NaN probs while `softmax` happily produces NaN from divergent logits; `tensor.norm()` second derivative blowing up without forward observable signal. None of these throw at the source of the problem. The codebase has to do the throwing.

4. **Hard gates are load-bearing.** When GNB silently absorbed multinomial (NaN) via a bare except RuntimeError, opt.step ran on NaN gradients for 30 steps before detection. The fix was three isfinite() checks, at logits, at CE loss, and per-param at $\hat{g}$, each raising with diagnostic context. The hard gates halted at the actual source rather than 30 steps downstream on a wrecked model.
5. **K=4 is the right cadence default.** Window-aligned with a typical K=4 TBPTT cadence, K=4 gives both variance reduction (4 independent Hutchinson samples per opt-step) and per-step cost amortization. K=1 is more correct in the abstract; K=4 is more correct given how the rest of a typical training loop is sequenced.

8. Configuration

Field	Default	Meaning
lr	6e-2	Sophia paper default
betas	(0.965, 0.99)	Sophia momentum β_1 ; Hessian-diagonal EMA β_2
shampoo_beta	0.95	Kronecker covariance EMA decay
eps	1e-12	Hessian denominator floor (Sophia clamp)
eps_adam	1e-8	Spread-cap safe_max floor + 1-D Yogi epsilon
eps_yogi	1e-3	1-D Yogi variance floor
rho	0.04	Sophia clip threshold
gamma	0.04	Sophia Hessian denominator scaling ($\gamma \cdot h $)
weight_decay	0.0	Decoupled weight decay (AdamW-style)
refresh_freq	10	U_L, U_R eigh refresh cadence
hessian_freq	4	Hessian-diagonal refresh cadence (K, TBPTT-window-aligned)
eigh_residual_threshold	0.5	L2 eigh acceptance threshold (per-side)
spread_cap	10.0	L1 row-spread bound (in rotated basis)
radam_enabled	True	L4 RAdam cold-start gate
initial_accumulator	1e-6	Initial value for momentum + hessian_diag_rot

Field	Default	Meaning
<code>hvp_strategy</code>	<code>"hutchinson"</code>	Curvature estimator. <code>"hutchinson"</code> (true diagonal, includes negative curvature) or <code>"gnb"</code> (Gauss-Newton fallback, positive semidefinite, no second derivatives)

9. Conclusion

RACASO decouples the calculation of curvature topology from the tracking of gradient coordinate history. By combining Kronecker-factored structural rotations with rigorous monolithic Hutchinson HVP sampling, RACASO ensures that second-order curvature alignment is mathematically rigorous, computationally viable, and resilient to the structural challenges of modern branching network topologies.

The optimizer is designed for failure modes that competing second-order methods do not survive: ill-conditioned Kronecker covariance under coupled-gradient regimes, Newton-Schulz convergence failure under sudden distribution shifts, and the silent failure modes PyTorch eager autograd exposes when the second-derivative path is naively integrated. The four-layer safety chain (L1 through L4) absorbs the first three. The L5 safe-skip absorbs the fourth, documented in Section 6 as a general class for any second-order-aware optimizer.

The engineering provenance in Section 7 is preserved because each wall was a property of the codebase plus framework interaction, not a property of Hutchinson itself. The fix at each wall was either small or out-of-scope-but-named. The recipe didn't survive contact with the codebase, but each contact taught the codebase what it needed. The diagnostic infrastructure built during that integration (stage trap, anomaly mode integration, site probe utility) is reusable for any future optimizer integration that hits the same operator-shape failure surface.

RACASO ships as a drop-in for any model with branching gradient paths and coupled-gradient regimes. The underlying composition (CASPR rotation plus Sophia cautious step in the rotated basis, with the four-layer safety chain and L5 absorption) is general.

License

MIT. See LICENSE file.